11-766: Large Language Model Applications Due 11:59 PM on 2/19

Project Proposal: Multi-Agent Financial RAG

Somaya Jain *somayaj*

1 Abstract

This project proposes a large-scale multi-agent retrieval-augmented generation (RAG) system for financial risk analysis and investment research. The system is designed to operate over publicly available financial data for the top 50 U.S. public companies by market capitalization, including SEC 10-K and 10-Q filings, earnings call transcripts, and financial news articles. The goal is to build an AI system capable of producing structured, citation-backed financial analysis comparable to an entry-level research analyst. Users interact with the system by asking questions such as "Assess the financial risk of Company X over the next year," "Compare liquidity and regulatory risk between Company A and Company B," or "How might rising interest rates affect this firm's debt exposure?" The system responds with a comprehensive analyst-style report containing extracted financial metrics, quantitative trend analysis, qualitative sentiment interpretation, categorized risk factors, and explicit supporting citations from source documents.

Instead of relying on a single RAG pipeline, the system decomposes analysis into specialized components. A planner agent interprets the user query and assigns subtasks. A retrieval agent selectively gathers relevant filings and transcripts using metadata filtering and semantic search. A financial metrics agent extracts and computes structured indicators such as revenue growth, leverage ratios, liquidity measures, and cash flow trends. A qualitative analysis agent evaluates management tone, forward-looking statements, and recurring risk language in transcripts. A risk synthesis agent integrates quantitative and qualitative evidence to categorize risks (e.g., market, credit, liquidity, regulatory, macroeconomic). Finally, a critic agent verifies that generated claims are grounded in retrieved documents and flags unsupported assertions, thereby reducing hallucination in a high-stakes domain.

2 Literature review

1. Analysis of "Enhancing Financial RAG with Agentic AI and Multi-HyDE" (arXiv:2509.16369)

This paper by Srinivasan et al. (2025) [Srinivasan et al., 2025] tackles a problem very close to what I'm working on — building a reliable RAG system for financial question-answering over SEC filings, earnings transcripts, and market data. What they do: They introduce a financial RAG framework that combines agentic AI with a Multi-HyDE system, which generates multiple non-equivalent queries to improve retrieval coverage from large financial corpora. Instead of sending a single query to the vector store, the system creates several reformulated queries that capture different aspects of the user's intent, addressing the common mismatch between short questions and long, detailed source documents.

The LLM acts as an orchestrator that decomposes complex queries, selects appropriate tools or retrieval strategies, performs multi-hop searches, and verifies intermediate results before generating a final answer. They combine this with hybrid retrieval BM25 keyword search, semantic search, and table-based extraction so the agent can pick the right tool depending on whether it needs a specific number from a table or a conceptual passage.

Results: Their pipeline improves accuracy by 11.2% and reduces hallucinations by 15% on benchmarks like ConvFinQA, FinanceBench, and financial-qa-10K.

We're both building agentic RAG systems over the same types of financial documents. However, my system differs in a few key ways. Their approach is essentially a single orchestrator agent choosing among tools, whereas I decompose the analysis across multiple specialized agents (planner, retrieval, financial metrics, qualitative analysis, risk synthesis, and critic). My output target is also different i.e. I'm generating full analyst style reports with structured risk categorization and citations, not just answering benchmark questions. That said, their Multi-HyDE technique is something I can incorporate into my retrieval agent to boost recall, and their finding that hybrid retrieval outperforms pure semantic search reinforces my decision to use metadata filtering alongside embeddings. I also plan to use some of the same benchmarks they evaluated on. One limitation they acknowledge is that even with improved retrieval, post-generation errors like calculation mistakes and temporal inconsistencies still persist. This is exactly why I've included a dedicated critic agent in my pipeline — to catch unsupported or incorrect claims after generation, which their system doesn't explicitly address.

2. Analysis of "Beyond Summaries: Multi-Agent Generation of Investment Reports with Text, Tables, and Charts" (Yang & Peng, EMNLP 2025 FinNLP Workshop)
This paper by Yang and Peng (2025) is closely relevant to my project because it directly addresses the problem of generating structured, analyst style investment reports from earnings call transcripts using a multi-agent system, which is essentially one of the core outputs my system aims to produce [Yang and Peng, 2025].

They build an AutoGen-based multi-agent pipeline for the Earnings2Insights shared task. Starting with 64 official transcripts, they expand the dataset to 207 by crawling additional quarters from public sources (Motley Fool, Alpha Street) to provide temporal context. Their system then uses five specialized agents — a Chart Agent (extracts financial metrics like EPS and revenue into structured JSON), a Highlight Agent (distills key insights around financial trends, strategic shifts, management tone), a Report Writer (drafts full reports with sections like Executive Summary, Investment Thesis, Risks), a Content Editor (verifies factual accuracy and logical coherence), and a Styling Agent (applies institutional formatting). They use Gemini-Pro 2.5 as the backbone LLM and incorporate RAG over the enriched transcript corpus.

Their human evaluation showed an average investment decision accuracy of 52.4% (with weekly predictions reaching 56.8%), and metric-based evaluation scored well on logic (5.39/7) and usefulness (5.4/7). The results are modest but demonstrate that multi-agent collaboration produces more structured, persuasive outputs than single-pass generation.

3 Industry review

AWS Multi-Agent Investment Research Assistant[AWS Blog, 2025]
AWS published a reference architecture (May 2025) for a multi-agent investment research assistant using Amazon Bedrock. It is the closest industry parallel to my project. The system uses a supervisor agent that decomposes user queries and delegates to three subagents: a Quantitative Analysis Agent (stock data and portfolio optimization), a News Agent (retrieves SEC filings and earnings reports via RAG, with web search fallback), and a Smart Summarizer Agent (synthesizes outputs into structured insights). The RAG pipeline uses Amazon Bedrock Data Automation for parsing multimodal data and OpenSearch Serverless for vector search, with Claude as the backbone LLM.

The supervisor-subagent pattern closely mirrors my planner agent delegating to specialized agents, and their agent roles map almost directly onto mine. However, their system lacks two components central to my design: a dedicated qualitative analysis agent for interpreting management tone and sentiment, and a critic agent for verifying that claims are grounded in source documents. Their summarizer consolidates outputs but performs no explicit hallucination checking. Their use of multimodal parsing (including audio earnings calls) and parallel agent invocation are practical ideas I may incorporate. The fact that AWS chose this exact use case as a flagship demo for their multi-agent platform validates that my problem space is both commercially relevant and well-suited to the architecture I'm proposing.

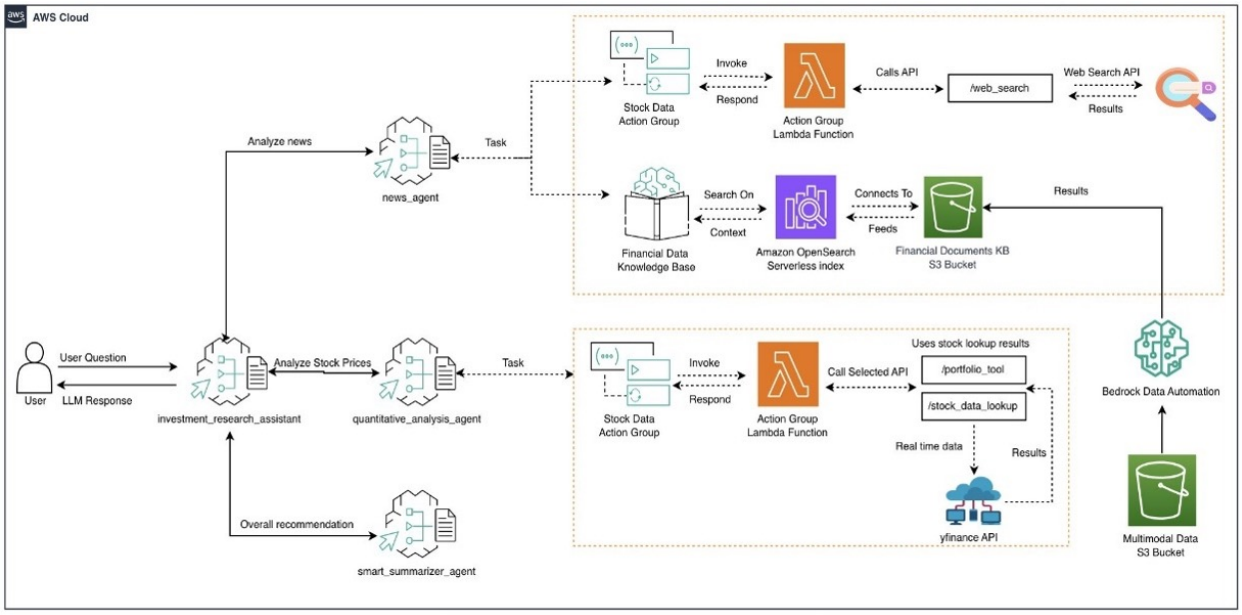

Figure 1: AWS reference architecture for a multi-agent investment research assistant (Amazon Bedrock).

Hebbia AI[Hebbia, 2025]

Hebbia builds Matrix, a document intelligence platform used by major investment banks and over 40% of the largest asset managers by AUM. It processes SEC filings, earnings transcripts, and

research reports, returning structured, citation-linked answers to complex queries across thousands of documents.

Their system uses multi-model routing, sending extraction tasks to models like Claude and GPT-4.1 while using reasoning models like o3 for summarization. This achieves 92% accuracy on financial document benchmarks, compared to 68% with standard RAG. The platform handles the same core tasks my system targets i.e. multi-document financial analysis, tracking management language changes across quarters, extracting metrics, and generating structured comparisons.

4 Mockup

The wireframe (Figure 2) shows a web-based dashboard where users select a company and time range, then ask financial analysis questions in natural language. The system responds with five structured panels mapping directly to our agent outputs:

- **Executive Summary**: Overall risk rating with narrative assessment (Risk Synthesis Agent).
- **Key Financial Metrics**: Extracted indicators with values, trends, and source links (Financial Metrics Agent).
- **Management & Sentiment Analysis**: Tone assessment and key themes from earnings calls (Qualitative Analysis Agent).
- **Risk Categorization**: Five risk types with severity levels, each expandable for evidence (Risk Synthesis Agent).
- **Verification Status**: Claim count verified against source documents (Critic Agent).

A chat panel allows follow-up questions within the report context, and results can be exported as PDF.

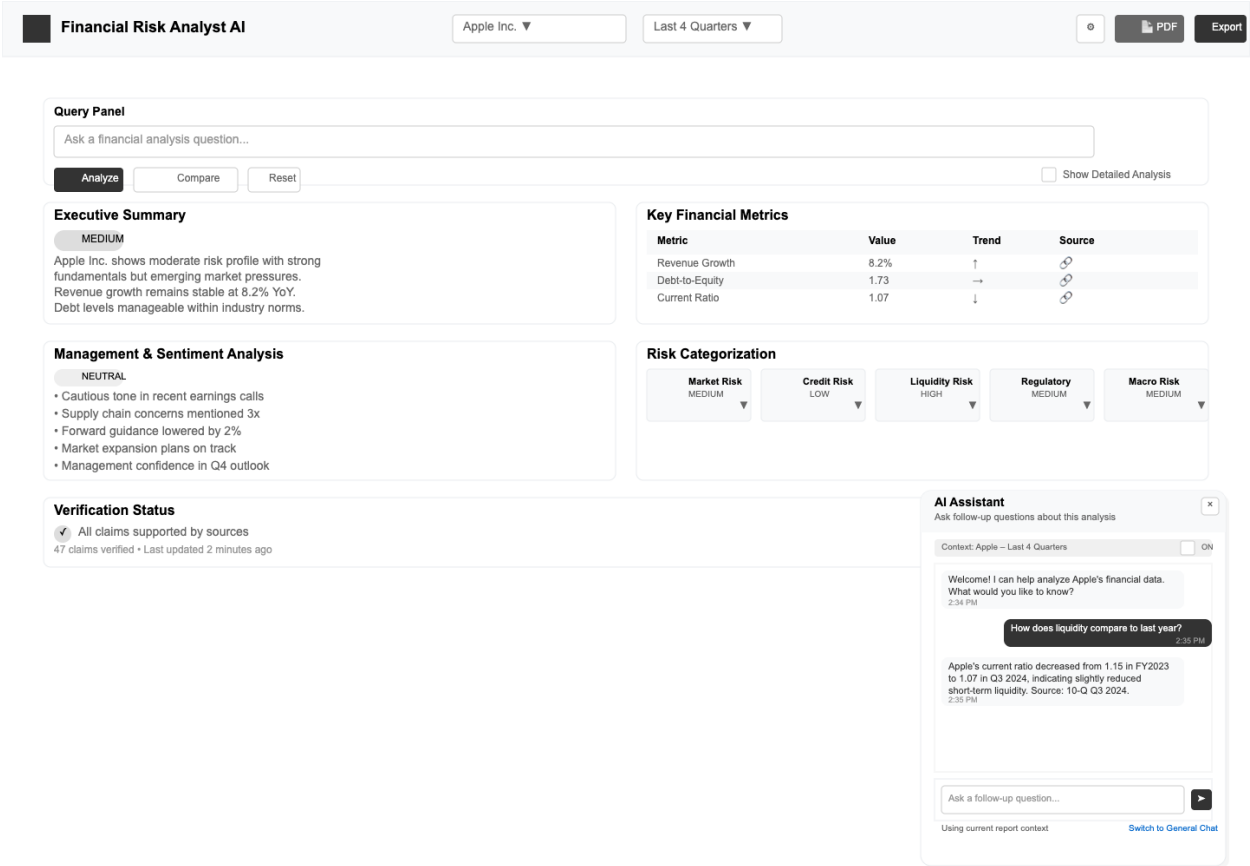

Figure 2: Wireframe mockup of the user interface and report output (illustrative only; the final UI may differ).

5 Proposed Method

System Architecture:

I propose to build a multi-agent retrieval-augmented generation (RAG) pipeline for financial risk analysis using GPT-5 as the backbone LLM, LangGraph for agent orchestration, FAISS for vector storage, and OpenAI embeddings for semantic representation. The system is designed to support structured, citation-backed financial analysis across the top 50 U.S. public companies by market capitalization.

Orchestration Layer:

Agent coordination will be implemented using LangGraph, modeling the workflow as a stateful directed graph. Each agent functions as a node, with conditional edges defining transitions based on the planner's execution logic. LangGraph's state management enables intermediate outputs (retrieved document chunks, extracted metrics, qualitative signals) to be passed between agents as shared state. This design supports both sequential and parallel execution. For example, the financial metrics agent and qualitative analysis agent will operate in parallel over retrieved documents,

with their outputs converging at the risk synthesis node. The critic agent may trigger conditional re-retrieval if insufficient evidence is detected.

Agent Specifications:

Planner Agent: The planner will generate a structured execution plan that determines the sequence of agents and required parameters. Initially, the system will support predefined query types (single-company analysis and cross-company comparison) to ensure stability and feasibility. As the system matures, the planner can be extended to support more flexible task graphs. For comparative queries (e.g., "Compare liquidity risk between Company A and Company B"), the planner will generate parallel retrieval subtasks before routing outputs to downstream agents.

Retrieval Agent:

The retrieval agent will operate over a FAISS vector index built from OpenAI embeddings of chunked financial documents (approximately 512 tokens with overlap), each tagged with metadata such as company ticker, filing type, fiscal period, and document section.

To reduce query–document mismatch, the agent will incorporate a Multi-HyDE-style query expansion mechanism. Instead of issuing a single query, the system will generate multiple reformulated queries capturing different aspects of the user's intent (e.g., numerical metrics, risk disclosures, qualitative explanations).

Retrieval will follow a hybrid approach. After metadata filtering narrows the search space (e.g., specific company and fiscal year), both keyword-based search (BM25) and embedding-based similarity search will be applied. Results from multiple reformulated queries and search methods will be merged, deduplicated, and reranked before passing the top-k chunks to downstream agents. This strategy improves recall and robustness when operating over long, structured financial documents.

Financial Metrics Agent: This GPT-5-powered agent will extract and standardize quantitative indicators from retrieved content using a structured output schema. Target metrics include revenue growth rate, debt-to-equity ratio, interest coverage ratio, current ratio, quick ratio, operating cash flow trends, and margin analysis. Outputs will be returned in structured JSON format with citations to source document chunks. Where machine-readable tables are available, numerical values will be extracted programmatically; otherwise, structured LLM-based extraction will be applied.

Qualitative Analysis Agent: This agent will analyze earnings call transcripts using domain-specific prompt templates. It will evaluate management tone, identify forward-looking statements, detect recurring risk themes across quarters, and highlight notable shifts in strategic emphasis. Quantitative sentiment scoring may be incorporated as a supplementary signal alongside GPT-5's interpretive analysis to provide measurable sentiment indicators.

Risk Synthesis Agent: This agent will integrate structured outputs from the metrics and qualitative agents. Identified risks will be categorized into five classes: market risk, credit risk, liquidity risk, regulatory risk, and macroeconomic risk. Each risk will be assigned a severity level (high/medium/low) based on the strength and consistency of supporting evidence. The agent will produce a unified, citation-backed risk profile.

Critic Agent: The critic agent will perform claim-level verification on the generated report. For each factual assertion or extracted metric, it will verify whether supporting evidence exists within retrieved document chunks. Unsupported claims will be flagged, and the system may either trigger re-retrieval or mark the statement as unverified.

Data Ingestion Pipeline:

All collected documents will undergo preprocessing including parsing (PDF/HTML), section segmentation, chunking with overlap, metadata tagging (company, filing type, fiscal period, section), embedding generation, and indexing into FAISS for efficient retrieval.

Datasets:

Data will be collected from the following sources:

- SEC EDGAR filings (10-K, 10-Q) via sec-api.io
- Earnings call transcripts from publicly available sources
- Financial news articles via news APIs (e.g., Finnhub Company news API)
- Benchmark datasets: FinanceBench and ConvFinQA

Evaluation Scheme:

The system will be evaluated across four dimensions:

Retrieval Quality: Recall@k and MRR@k will measure whether the retrieval agent surfaces relevant source documents.

Answer Accuracy: Extracted financial metrics will be compared against ground-truth values from filings. Faithfulness scoring will verify that generated claims correspond to retrieved source chunks.

Hallucination Detection: The critic agent's ability to identify unsupported claims will be measured using precision and recall of flagged assertions.

Report Quality: Generated reports will be evaluated for clarity, coherence, completeness of risk coverage, and citation quality using LLM-as-judge scoring methodology.

6 Midpoint goals

Expected Progress:

- Data pipeline operational: SEC filings and earnings transcripts for the top 50 companies downloaded, parsed, chunked, embedded with FinBERT, and indexed in FAISS with metadata tags.
- Retrieval Agent working with two-stage retrieval (metadata filtering + semantic search).
- Planner Agent routing single-company queries to downstream agents via LangGraph.
- Financial Metrics Agent extracting core indicators (leverage ratios, liquidity measures, revenue growth) as structured JSON from retrieved chunks.
- Qualitative Analysis Agent producing sentiment assessments from earnings transcripts using GPT-5 and OpenAI embeddings.

- End-to-end LangGraph pipeline connecting Planner, Retrieval, Metrics, and Qualitative agents for single-company queries.

Reach Goals:

- Risk Synthesis Agent integrated and producing categorized risk profiles from combined agent outputs.
- Critic Agent performing basic claim-level verification on generated reports.
- Support for comparative multi-company queries.
- Initial evaluation against FinanceBench or ConvFinQA benchmarks.

7 End-of-Semester goals

Expected Progress (Full Marks):

- All six agents (Planner, Retrieval, Financial Metrics, Qualitative Analysis, Risk Synthesis, Critic) fully implemented and integrated in the LangGraph pipeline.
- System handles both single-company risk analysis and comparative multi-company queries end-to-end.
- Risk Synthesis Agent producing structured risk profiles with categorized risks (market, credit, liquidity, regulatory, macroeconomic) and severity indicators backed by evidence from both quantitative and qualitative agents.
- Critic Agent performing claim-level verification, flagging unsupported assertions, and demonstrably reducing hallucination in generated reports.
- Final output is a structured, citation-backed analyst-style report containing executive summary, financial metrics, trend analysis, risk categorization, and source references.
- Full evaluation completed: retrieval quality (Recall@k, MRR@k), answer accuracy against ground truth, hallucination detection precision/recall, and report quality scoring via LLM-as-judge.
- Benchmarking against FinanceBench and ConvFinQA with reported results.
- Clear documentation of system architecture, design decisions, and evaluation methodology.

Reach Goals:

- Multi-model routing (e.g., using OpenAI embeddings for extraction and GPT-4o for reasoning) optimized per agent to improve accuracy and reduce cost, inspired by Hebbia's approach.
- Temporal trend analysis across multiple quarters, automatically detecting shifts in risk language or financial trajectory over time.

- Interactive user interface where analysts can ask follow-up questions and drill down into specific risk categories or metrics.
- Integration of a graph-based data layer (e.g., Neo4j or ArangoDB) alongside FAISS to model corporate relationships and sector linkages for richer comparative analysis.
- Real-time ingestion of new filings and transcripts with automatic re-indexing and alert generation for material risk changes.

8 Engagement with course material

This project implements a full retrieval-augmented generation pipeline that separates parametric knowledge (LLM weights) from non-parametric knowledge (external financial documents). Instead of relying on static model memory, the system retrieves SEC filings, earnings transcripts, and financial news and incorporates them into generation to produce citation-backed, verifiable outputs. The retrieval layer combines sparse keyword-based search (BM25) and dense embedding-based retrieval to balance lexical precision and semantic matching. Query expansion and hybrid retrieval are used to reduce query–document mismatch and improve recall, moving beyond naïve embedding-only approaches.

The architecture also addresses common failure modes of in-context RAG, such as hallucinated or weakly supported claims, through a dedicated critic agent that performs claim-level grounding verification. Evaluation is conducted using both intrinsic metrics (e.g., Recall@k for retrieval quality) and extrinsic metrics (end-to-end factual accuracy and faithfulness). In addition, LLM-as-Judge evaluation is used to assess report coherence, logical consistency, risk coverage completeness, and citation alignment, providing a scalable framework for qualitative evaluation. Together, these components demonstrate the application of advanced retrieval modeling, multi-agent orchestration, verification mechanisms, and systematic evaluation beyond simple prompt-based LLM usage.

9 Use of AI

AI tools were used during the ideation and planning stages of this project. Specifically, AI assistance was used for brainstorming application features, identifying appropriate agent roles for the multi-agent workflow, and comparing orchestration frameworks (e.g., LangGraph, AutoGen, CrewAI) to determine a suitable architecture. AI was also used to explore trade-offs between embedding models and to better understand retrieval and RAG design choices relevant to the project domain.

Additionally, AI was used for limited paraphrasing and refinement of written sections to improve clarity and organization. All architectural decisions, system design choices, and technical content were critically reviewed and finalized independently.

References

AWS Blog. Building an ai-powered assistant for investment research with multi-agent collaboration in amazon bedrock and amazon bedrock data automation. https://aws.amazon.com/blogs/machine-learning/part-3-building-an-ai-powered-assistant-for-investment-research-with-multi-agent-collaborati 2025.

Hebbia. Hebbia: Ai for finance. https://www.hebbia.com, 2025.

Akshay Govind Srinivasan, Ryan Jacob George, Jayden Koshy Joe, Hrushikesh Kant, Harshith M R, Sachin Sundar, Sudharshan Suresh, Rahul Vimalkanth, and Vijayavallabh. Enhancing financial rag with agentic ai and multi-hyde: A novel approach to knowledge retrieval and hallucination reduction, 2025. URL https://arxiv.org/abs/2509.16369.

Weijie Yang and Junbo Peng. Beyond summaries: Multi-agent generation of investment reports with text, tables, and charts. In Chung-Chi Chen, Genta Indra Winata, Stephen Rawls, Anirban Das, Hsin-Hsi Chen, and Hiroya Takamura, editors, *Proceedings of The 10th Workshop on Financial Technology and Natural Language Processing*, pages 252–257, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.finnlp-2.18. URL https://aclanthology.org/2025.finnlp-2.18/.